VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

A Database Management System Mini Project Report on

DevMatch AI – Developer Portfolio & Hiring Platform

Submitted in Partial fulfillment of the Requirements for the VII Semester of the Degree of

Bachelor of Engineering in

Electronics and Communication Engineering

By

Name _____ **USN** _____

Name _____ **USN** _____

Name _____ **USN** _____

Under the Guidance of,

Guide name _____ **Designation** _____

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with "A++" Grade

ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

2026-27

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with "A++" Grade

ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the Database Management System Project work entitled "DevMatch AI – Developer Portfolio & Hiring Platform" has been carried out by _____, USN _____, _____, USN _____, and _____, USN _____, bonafide students of CMR Institute of Technology, Bengaluru in partial fulfillment for the award of the Degree of Bachelor of Engineering in Electronics and Communication Engineering of the Visvesvaraya Technological University, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. This Database Management System Project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said Degree.

_____	_____
Signature of Guide	Signature of HOD
Name _____	Name _____
Designation	Designation
_____	_____

DECLARATION

We, the students of VII semester from the Department of Electronics and Communication Engineering, CMR Institute of Technology, Bangalore declare that the project work entitled "DevMatch AI – Developer Portfolio & Hiring Platform" has been successfully completed under the guidance of _____, designation _____, CMR Institute of Technology, Bengaluru. This project work is submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Electronics and Communication Engineering during the academic year 2026-2027. The matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Place: _____

Date: _____

Team members:

Name _____ **USN** _____	_____
Name _____ **USN** _____	_____
Name _____ **USN** _____	_____

ABSTRACT

DevMatch AI is a database driven developer portfolio and hiring platform. Developers manage profiles, skills, projects, resumes and applications, while recruiters publish jobs and review candidates. Flask and SQLAlchemy connect a responsive web interface to ten related MySQL tables. Normalized skill and technology records, integrity constraints, joins, views and a status audit trigger demonstrate DBMS concepts. PDF resume extraction supports transparent skill matching, with optional Gemini analysis. The system centralizes hiring information and allows recruiter review of automatically assigned application statuses.

Keywords: MySQL, Flask, Database Normalization, Developer Portfolio, Skill Matching

ACKNOWLEDGEMENT

We express our gratitude to CMR Institute of Technology, Bengaluru, for providing the facilities and academic environment to carry out this Database Management System mini project.

We thank _____, Principal, CMR Institute of Technology, Bengaluru, for encouragement and institutional support.

We express our gratitude to _____, Head of the Department of Electronics and Communication Engineering, CMR Institute of Technology, Bengaluru, for support throughout the project.

We thank our guide _____, designation _____, CMR Institute of Technology, Bengaluru, for guidance during the project work.

We thank our parents, friends and team members for their support and cooperation during the completion of this project.

TABLE OF CONTENTS

Contents	Page No.
Certificate	ii
Declaration	iii
Abstract	iv
Acknowledgement	v
Table of Contents	vi
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Objectives and 1.2 Scope of the project	2
2 System Requirements	3
2.1 Front End Tools and 2.2 Back End Tools	3
3 Design	4
3.1 Schema Diagram	4
3.2 ER Diagram	5
3.3 Database Tables	6–9
4 Implementation	10
4.1 Installation Steps	10
4.2 Front-end Code	11
4.3 Back-end Code	12–15
5 Interpretation of Result	16
5.1 Result Snapshots	16–18
6 Conclusion and Future Scope	19
References	20

LIST OF FIGURES

Figure	Page No.
Fig 3.1 Relational schema	4
Fig 3.2 Entity relationships and cardinalities	5
Fig 5.1 Developer dashboard	16
Fig 5.2 Job listing and skill match	17
Fig 5.3 Recruiter applicant review	18

LIST OF TABLES

Table	Page No.
Table 2.1 Development requirements	3
Table 3.1 Account and profile tables	6
Table 3.2 Portfolio tables	7
Table 3.3 Job tables	8
Table 3.4 Application and analysis tables	9

Chapter 1

INTRODUCTION

DevMatch AI is a web application that brings developer portfolios, job postings and application tracking into one relational database. A developer can present skills and projects, upload a resume, search jobs and monitor applications. A recruiter can maintain a company profile, publish vacancies and examine the evidence behind each candidate's skill match.

The project addresses the difficulty of keeping hiring information consistent when profiles, resumes, vacancies and application decisions are handled separately. Repeated candidate details and unstructured skill lists make searching and comparison difficult. DevMatch AI stores related information in dedicated MySQL tables and links records through primary and foreign keys.

The system uses Flask for request handling, SQLAlchemy for database access and HTML, CSS and JavaScript for presentation. PDF text extraction provides resume evidence. Gemini can optionally produce a structured analysis, while a deterministic extraction path allows the resume workflow to function without an AI service. Supabase is used only for optional private resume file storage.

The DBMS contribution is the design and implementation of ten application tables, normalized skill and project technology records, uniqueness constraints and relational queries. A separate SQL demonstration adds a view and an application status audit trigger. The report explains the requirements, design, implementation and observed local results of these components.

Skill matching is intended to support review. The score measures coverage of a job's recorded skill names, and it does not measure overall competence or predict hiring success. The recruiter can revise the initial application status after examining the portfolio and resume.

1.1 Objectives

• Maintain developer and recruiter accounts with role specific access to their records.

• Store portfolios, skills, jobs, applications and resume analyses in related MySQL tables.

• Normalize multivalued job skills and project technologies to support searching and integrity.

• Provide create, read, update and delete operations through the web interface.

• Calculate a visible skill match using profile skills and the latest extracted resume skills.

• Demonstrate keys, joins, filtering, grouping, views, transactions and a status history trigger.

1.2 Scope of the project

The implemented scope includes registration and login, profile maintenance, developer portfolio management, job search, recruiter job management, one application per developer and job, PDF resume analysis, candidate discovery and application status review. The DBMS demonstration page displays current job records from the same MySQL database used by the application.

Chapter 2

SYSTEM REQUIREMENTS

The system requires a browser, a Python application environment and a reachable MySQL database. Optional services extend resume analysis and deployment; the core profile, job and application functions use the same relational backend in both local and hosted configurations.

2.1 Front End Tools

HTML defines pages and forms; CSS provides responsive layouts, navigation, dashboards and job cards. Jinja templates bind server supplied records to the interface. JavaScript supports interactions such as copying job information and opening skill details. Form actions and database updates are processed by Flask rather than by browser access to MySQL.

2.2 Back End Tools

Python and Flask implement routes, sessions and role checks. Flask SQLAlchemy defines models and database transactions; PyMySQL supplies the MySQL connection. PyMuPDF extracts PDF text, requests supports Gemini API calls, and python dotenv loads local configuration. Flask Limiter throttles selected account actions. MySQL Workbench or the command line client supports SQL demonstrations. References [1]–[6] identify the relevant official documentation.

Table 2.1 Development requirements

Component	Requirement or role
Hardware	A computer capable of running Python, MySQL and a modern browser; 4 GB RAM and free storage are practical local starting points.
Software	Python 3.10 or later; MySQL 8; a browser; packages listed in requirements.txt.
Database	mysql+pymysql connection URL; dedicated database and account.
Optional services	Gemini API key; private Supabase resume bucket; Redis compatible limiter storage for production.

Chapter 3

DESIGN

The design separates shared account information from developer and recruiter data. Jobs belong to recruiters, while applications connect developers to jobs. Repeated skill and technology values are represented by child rows. Figure 3.1 summarizes the key links in the physical schema.

3.1 Schema Diagram

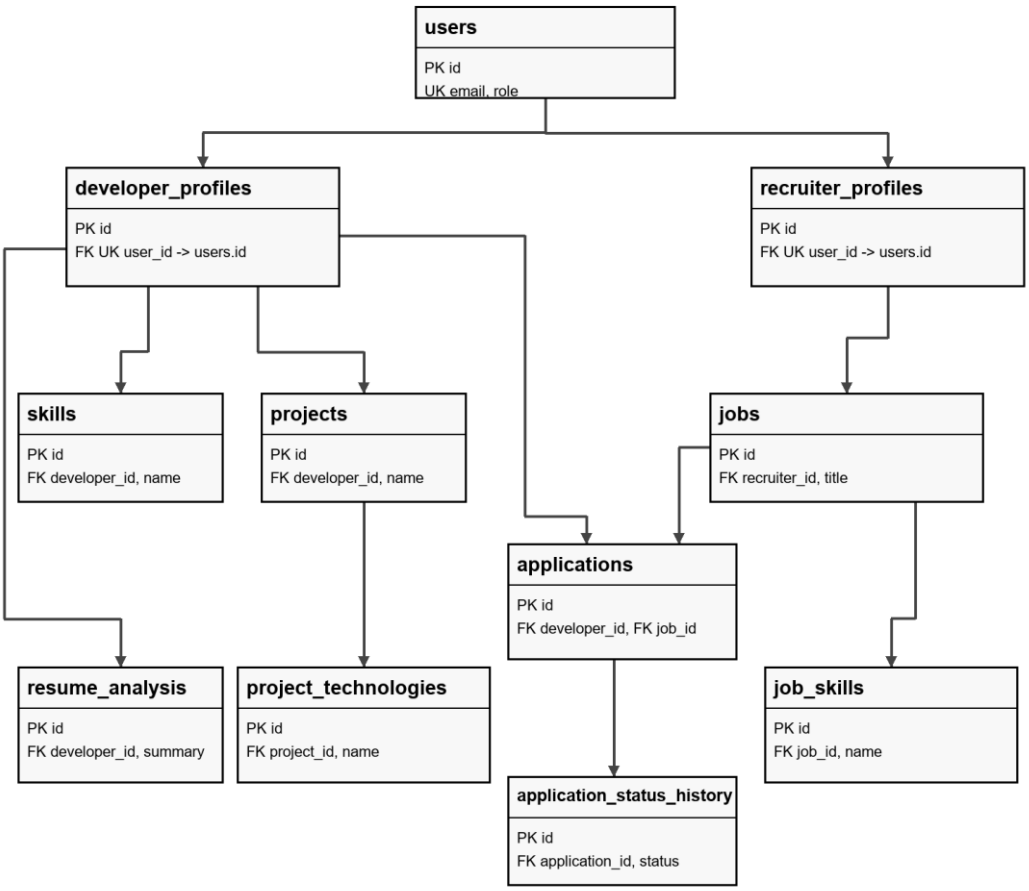


Fig 3.1 Relational schema

PK denotes a primary key, FK a foreign key and UK a unique key. Ten tables are created by the application models. The eleventh table, application_status_history, is an audit extension installed by mysql_demo.sql. The unique developer and job pair in applications prevents duplicate applications.

3.2 ER Diagram

Figure 3.2 shows the entity relationships and their cardinalities. Each profile references exactly one user, and a user has at most one profile of each kind. The application's role logic selects the appropriate profile. Child entities can have zero or many records for their parent.

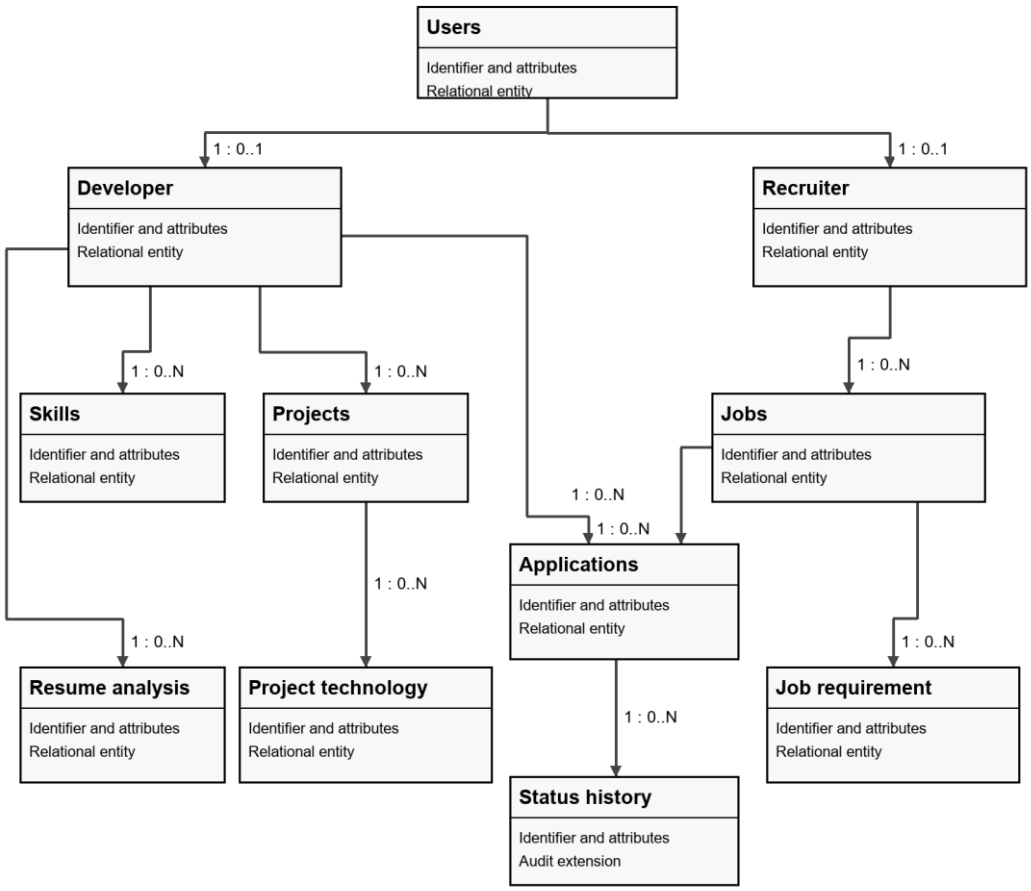


Fig 3.2 Entity relationships and cardinalities

The developer to job relationship is many to many and is resolved by Application. Each Application belongs to one Developer and one Job. Each Resume analysis and Skill belongs to one Developer. Each Job requirement belongs to one Job. The audit extension stores status transitions for an application.

3.3 Database Tables

Tables 3.1–3.4 describe the actual SQLAlchemy model columns. VARCHAR lengths are shown as String lengths in the model. Integer identifiers are surrogate primary keys. The profile user_id columns are unique foreign keys, so a user cannot have duplicate records in the same profile table.

Table 3.1 Account and profile tables

Table	Attribute	Type	Key or role
users	id	Integer	PK
users	name	String(120)	Attribute
users	email	String(190)	UNIQUE
users	password_hash	String(255)	Attribute
users	role	String(20)	Attribute
users	created_at	DateTime	Attribute
developer_profiles	id	Integer	PK
developer_profiles	user_id	Integer	FK users.id, UNIQUE
developer_profiles	bio	Text	Attribute
developer_profiles	phone	String(30)	Attribute
developer_profiles	location	String(100)	Attribute
developer_profiles	education	String(160)	Attribute
developer_profiles	experience	String(160)	Attribute
developer_profiles	github_url	String(250)	Attribute
developer_profiles	linkedin_url	String(250)	Attribute
developer_profiles	resume_path	String(255)	Attribute
recruiter_profiles	id	Integer	PK
recruiter_profiles	user_id	Integer	FK users.id, UNIQUE
recruiter_profiles	company_name	String(140)	Attribute
recruiter_profiles	company_description	Text	Attribute
recruiter_profiles	location	String(100)	Attribute
recruiter_profiles	website	String(250)	Attribute

Email is unique and the stored credential is a password hash. Phone and web links remain strings. User credentials are stored once and are referenced by the role specific profile.

3.3 Database Tables continued

Portfolio records are separated into skills, projects and project technologies. The developer owns each skill and project, and each technology belongs to a project. This avoids storing a comma separated technology list in the projects table.

Table 3.2 Portfolio tables

Table	Attribute	Type	Key or role
skills	id	Integer	PK
skills	developer_id	Integer	FK developer_profiles.id
skills	name	String(80)	Attribute
skills	level	String(30)	Attribute
skills	years	Float	Attribute
projects	id	Integer	PK
projects	developer_id	Integer	FK developer_profiles.id
projects	name	String(140)	Attribute
projects	description	Text	Attribute
projects	github_link	String(250)	Attribute
projects	project_link	String(250)	Attribute
project_technologies	id	Integer	PK
project_technologies	project_id	Integer	FK projects.id
project_technologies	name	String(80)	Attribute
project_technologies	category	String(40)	Attribute
project_technologies	version	String(40)	Attribute
project_technologies	added_at	DateTime	Attribute

The skills table has a uniqueness constraint on developer_id and name. Project technologies are unique on project_id and name. SQLAlchemy relationships provide access to these child rows, and form inputs are parsed into distinct entries before saving. The display properties can join values for presentation without reverting to a multivalued database column.

3.3 Database Tables continued

Jobs contain vacancy details while job_skills stores one named requirement per row. The recruiter_id foreign key connects each vacancy to its owner. Table 3.3 lists the job columns and requirement attributes.

Table 3.3 Job tables

Table	Attribute	Type	Key or role
jobs	id	Integer	PK
jobs	recruiter_id	Integer	FK recruiter_profiles.id
jobs	title	String(150)	Attribute
jobs	description	Text	Attribute
jobs	location	String(100)	Attribute
jobs	salary	String(100)	Attribute
jobs	experience_required	String(100)	Attribute
jobs	job_type	String(40)	Attribute
jobs	deadline	Date	Attribute
jobs	created_at	DateTime	Attribute
job_skills	id	Integer	PK
job_skills	job_id	Integer	FK jobs.id
job_skills	name	String(80)	Attribute
job_skills	importance	String(20)	Attribute
job_skills	minimum_years	Float	Attribute
job_skills	added_at	DateTime	Attribute

The job_id and name pair is unique in job_skills. Each job is linked to its recruiter through recruiter_id, and each requirement is linked to its job through job_id. These relationships support job management and skill matching without storing comma separated skill values in the jobs table.

3.3 Database Tables continued

Applications resolve the developer and job association. Resume analyses hold extracted information used by the matching function. Status history is a separate SQL audit table, created only when the demonstration script is run.

Table 3.4 Application and analysis tables

Table	Attribute	Type	Key or role
applications	id	Integer	PK
applications	developer_id	Integer	FK developer_profiles.id
applications	job_id	Integer	FK jobs.id
applications	status	String(30)	Attribute
applications	created_at	DateTime	Attribute
resume_analysis	id	Integer	PK
resume_analysis	developer_id	Integer	FK developer_profiles.id
resume_analysis	summary	Text	Attribute
resume_analysis	strengths	Text	Attribute
resume_analysis	missing_skills	Text	Attribute
resume_analysis	extracted_skills	Text	Attribute
resume_analysis	created_at	DateTime	Attribute
application_status_history	id	INT	PK
application_status_history	application_id	INT	FK applications.id
application_status_history	old_status	VARCHAR(30)	Nullable
application_status_history	new_status	VARCHAR(30)	Required
application_status_history	changed_at	DATETIME	Timestamp

Chapter 4

IMPLEMENTATION

The implementation connects browser forms to Flask routes, validates the request, reads or changes SQLAlchemy models and commits successful changes to MySQL. Jinja templates render the resulting records. The modules include accounts, portfolios, recruiter jobs, applications, resume analysis, skill matching and database demonstrations.

4.1 Installation Steps for Execution of the Project

Create a virtual environment in the devmatch-ai folder and install the project requirements. Create a MySQL database named devmatch_ai and an application user with access to that database. Copy .env.example to .env and set DATABASE_URL to the MySQL connection URL and SECRET_KEY to a private random value. Keep credentials outside source code.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
flask --app app init-db
flask --app app seed-demo
flask --app app verify-dbms
flask --app app run --debug
```

Open the local Flask address in a browser. Run mysql_demo.sql in the same MySQL database to create the summary view, audit table and status trigger. The seed command supplies demonstration accounts and records; it should be used in a dedicated demo database. The verifier checks table rows, attributes and relationships.

4.2 Front-end Code

The front end is organized as Jinja templates extending base.html. Job cards are rendered from the current query results. Each application form includes a CSRF token, and developer only actions are conditionally displayed. The following excerpts come from jobs.html, with line wrapping adjusted for the report.

```
{% for job in jobs %}
<article class="job-row">
  <div class="small-muted">
    {{job.recruiter.company_name}} · {{job.location}}
  </div>
  <h2>{{job.title}}</h2>
  <p>{{job.description}}</p>
  {% for skill in job.required_skills.split(',') %}
    <span>{{skill.strip()}}</span>
  {% endfor %}
</article>
{% endfor %}
```

The apply form submits to the Flask route for the selected job. The server rechecks identity, eligibility and existing applications; conditional rendering alone is not used as the access control.

```
<form method="post"
      action="{{url_for('apply',id=job.id)}}">
  <input type="hidden" name="csrf" value="{{csrf}}">
  <button class="button small">Apply now ↗</button>
</form>
```

The developer dashboard summarizes skills, projects and applications, while the recruiter dashboard exposes job management and applicant review. Search forms use query parameters for role or skill and location. JavaScript enhances interaction, and responsive CSS adapts the page layout to available screen width.

4.3 Back-end Code

The backend defines the ten application tables as SQLAlchemy models. Password hashing is used for credentials. Session checks restrict authenticated workflows, and role checks protect developer and recruiter routes. Record ownership is checked before profile, job, skill or application changes.

Account and portfolio modules

```
# Login module excerpt
user = User.query.filter_by(email=email).first()
if user and check_password_hash(user.password_hash, password):
    session.clear()
    session['uid'] = user.id
    return redirect(url_for('dashboard'))

# Profile module excerpt
profile = (user.developer if user.role == 'developer'
           else user.recruiter)
for field in fields:
    value = request.form.get(field, '').strip()
    setattr(profile, field, value)
db.session.commit()
```

Registration validates account information and creates the role specific profile. Login checks the password hash and stores the authenticated user in the session. The developer portfolio module manages profile fields, skills and projects; technologies entered in a form are stored in project_technologies child rows. Recruiter profile fields describe the company rather than duplicating the shared credential record.

Application model and uniqueness

```
class Application(db.Model):
    __tablename__ = 'applications'
    id = db.Column(db.Integer, primary_key=True)
    developer_id = db.Column(db.Integer,
        db.ForeignKey('developer_profiles.id'), nullable=False)
    job_id = db.Column(db.Integer,
        db.ForeignKey('jobs.id'), nullable=False)
    status = db.Column(db.String(30), default='Applied',
                       nullable=False)
    created_at = db.Column(db.DateTime,
        default=lambda: datetime.now(timezone.utc))
    __table_args__ = (UniqueConstraint('developer_id',
        'job_id', name='uq_application'),)
```

4.3 Back-end Code continued

The job module allows recruiters to create, edit and remove their own postings. Required skill values are stored as job_skills child records. The application module checks whether the job exists, whether its deadline has passed and whether the developer already applied. A successful submission creates an application and commits it to MySQL.

Skill matching and initial status

Let R be the set of case normalized job skill names and C the union of profile skill names and the latest extracted resume skills. The displayed score is round(100 × |R ∩ C| / |R|), or zero when R is empty. The shortlist decision uses the unrounded fraction and a strict threshold above 70 percent.

```
# Skill and project module excerpts
db.session.add(Skill(developer_id=current_user().developer.id,
    name=name, level=request.form.get('level','Intermediate'),
    years=max(0,min(50,float(request.form.get('years') or 0)))))
db.session.commit()
project.technology_items = [ProjectTechnology(name=technology)
    for technology in parse_list_values(
        request.form.get('technologies',''))]
db.session.add(project)
db.session.commit()

# Matching and application module excerpt
def should_shortlist(matched_count, required_count):
    return (required_count > 0 and
        100 * matched_count > 70 * required_count)
result = ats_match(developer, job)
status_value = ('Shortlisted' if
    result['recommend_shortlist'] else 'Rejected')
db.session.add(Application(job_id=id,
    developer_id=developer.id, status=status_value))
db.session.commit()
```

A candidate matching three of three skills scores 100 percent and meets the threshold; two of three scores 67 percent and does not. Exactly seven of ten skills produces 70 percent and is rejected initially. A job with no recorded skills also does not meet the threshold. The recruiter can change the status to Applied, Under Review, Shortlisted, Rejected or Selected after review.

The function exposes matched, missing, profile matched and resume matched skill lists. These lists explain the score. Matching is based on recorded names, not semantic equivalence, and does not use proficiency levels, years, minimum experience or protected personal characteristics.

4.3 Back-end Code continued

Resume analysis and storage modules

The resume route accepts a PDF within the configured 5 MB request limit, sanitizes the filename and extracts document text using PyMuPDF. Structured analysis is saved as a ResumeAnalysis row linked to the developer. The matching function uses the latest analysis rather than combining every historical upload.

```
# Resume analysis module excerpt
document = pymupdf.open(stream=data, filetype='pdf')
text = '\n'.join(page.get_text() for page in document)[:25000]
analysis = analyze_resume(text)
db.session.add(ResumeAnalysis(developer_id=developer.id,
    summary=analysis['summary'],
    strengths=', '.join(analysis['strengths']),
    missing_skills=', '.join(analysis['missing_skills']),
    extracted_skills=', '.join(analysis['skills'])))
db.session.commit()

# Job management module excerpt
job.skills = [JobSkill(name=skill) for skill in required]
db.session.add(job)
db.session.commit()
job = Job.query.filter_by(id=id,
    recruiter_id=current_user().recruiter.id).first_or_404()
db.session.delete(job)
db.session.commit()
```

When a Gemini API key is configured, extracted resume text is submitted for structured analysis. Without a key or when the analysis request fails, the local extraction path identifies supported skills and returns a deterministic result. The career advisor uses the developer's profile and current job listings as context. It requires the configured AI service.

Local development can store PDF files in the upload directory. Production configuration uses a private Supabase bucket, and the authenticated resume route retrieves the file. MySQL stores the application records and resume path; Supabase's PostgreSQL database is not used by this project.

4.3 Back-end Code continued

SQL queries view and trigger

The supplied mysql_demo.sql demonstrates joins, grouping, filtering, a view and status auditing. The following view combines job ownership with application counts, including jobs with no applicants because the application join is a LEFT JOIN.

```
CREATE OR REPLACE VIEW job_application_summary AS
SELECT j.id AS job_id, j.title, r.company_name,
       COUNT(a.id) AS applicant_count,
       SUM(CASE WHEN a.status = 'Shortlisted'
                THEN 1 ELSE 0 END) AS shortlisted_count
FROM jobs j
JOIN recruiter_profiles r ON r.id = j.recruiter_id
LEFT JOIN applications a ON a.job_id = j.id
GROUP BY j.id, j.title, r.company_name;
```

The status trigger creates an audit row only when an UPDATE changes the application status. The SQL script separately backfills initial status records for existing applications. It does not install an INSERT trigger for every future application.

```
DELIMITER $$
CREATE TRIGGER log_application_status
AFTER UPDATE ON applications FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status THEN
    INSERT INTO application_status_history
      (application_id, old_status, new_status)
    VALUES (NEW.id, OLD.status, NEW.status);
  END IF;
END$$
DELIMITER ;
```

The script also uses ALTER TABLE to widen the salary column and demonstrates application INSERT, UPDATE and DELETE in a transaction ending in ROLLBACK. This keeps the transaction's test application from persisting. Website CRUD actions commit their changes; a new request reads updated records. The /dbms-demo page queries MySQL again and disables browser caching.

Chapter 5

INTERPRETATION OF RESULT

This chapter presents the main results of DevMatch AI. The snapshots show developer portfolio management, job search and skill matching, and recruiter review of applications. Each figure is followed by an interpretation of the displayed result.

5.1 Result Snapshots

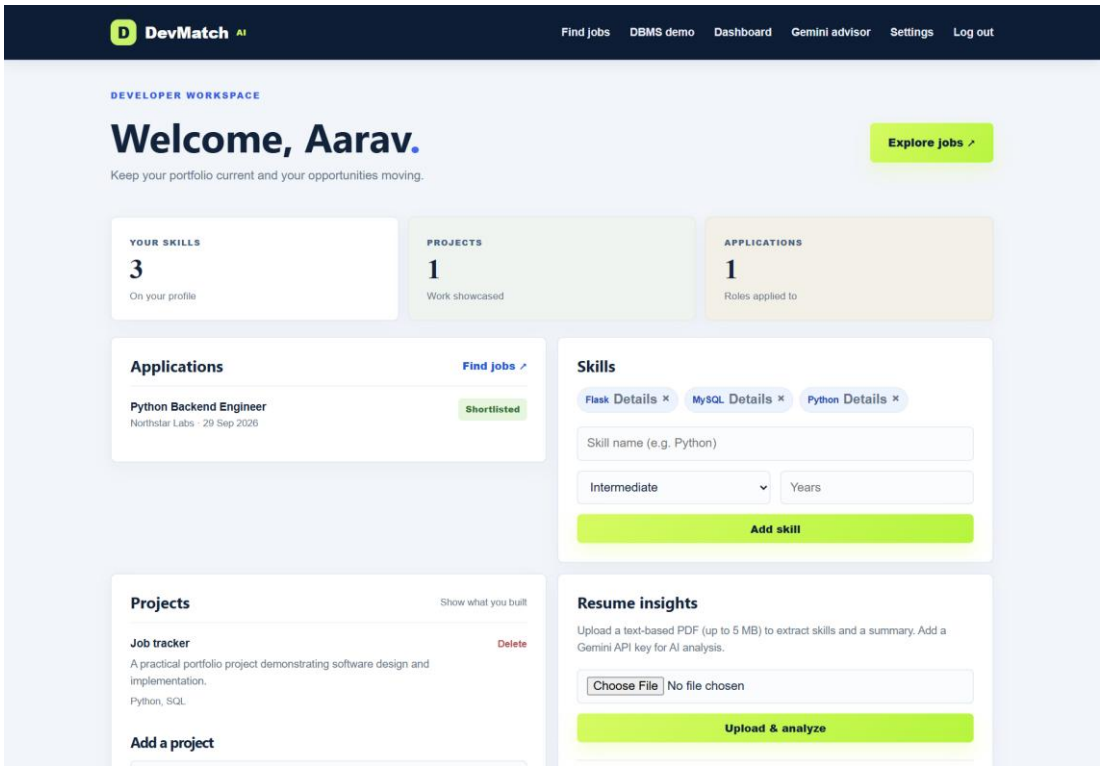


Fig 5.1 Developer dashboard

Interpretation: The developer dashboard displays the stored profile, skills, projects and applications. The sample developer has three skills, one project and one application. The application status is displayed alongside the job title. Skills and projects are stored in child tables linked to the developer profile, while resume analysis is linked separately. This result shows that portfolio information and application tracking are brought together through relational database queries.

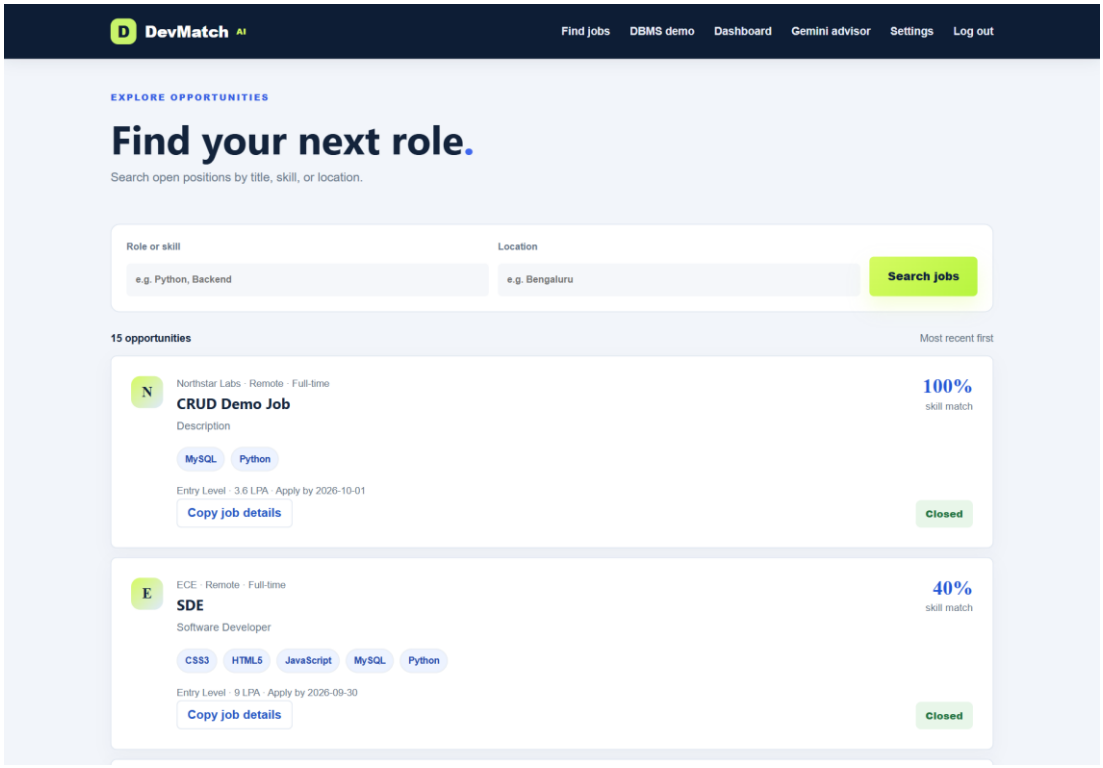


Fig 5.2 Job listing and skill match

Interpretation: The job page displays vacancies with company details, location, required skills and the developer's skill match. Search fields filter the current database records. The match is calculated from the intersection of recorded job skills with profile skills and the latest extracted resume skills. The application action depends on login, the closing date and whether an application already exists. This result connects the jobs, job_skills, recruiter_profiles and applications tables in one workflow.

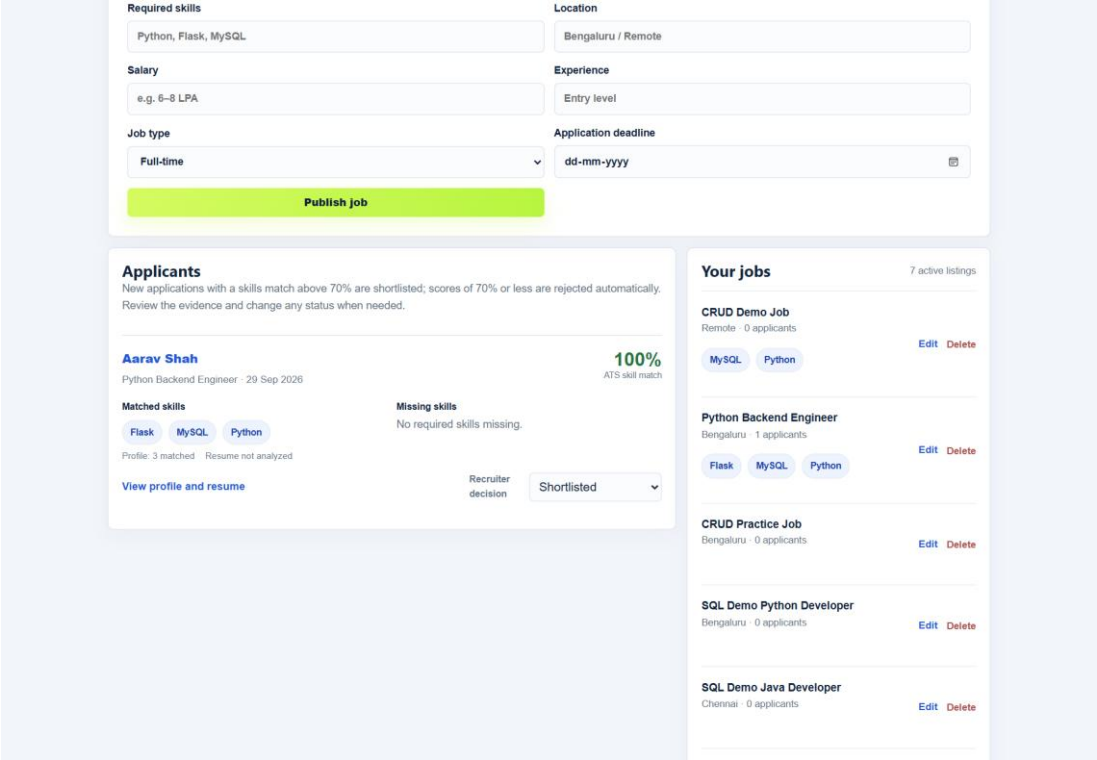


Fig 5.3 Recruiter applicant review

Interpretation: The recruiter dashboard displays applicants for the recruiter's own jobs. The illustrated candidate matches Python, Flask and MySQL, giving a 100 percent skill match. The screen identifies the profile as the evidence source and states that the resume has not been analyzed. Matched and missing skills help the recruiter interpret the score. The status control allows the recruiter to review and revise the application decision, which is saved to the applications table.

Chapter 6

CONCLUSION AND FUTURE SCOPE

DevMatch AI implements a relational developer portfolio and hiring workflow with separate developer and recruiter accounts. The database connects profiles, skills, projects, vacancies, applications and resume analyses. The interface gives users a consistent way to maintain their information and gives recruiters a visible record of candidate skill matches and application decisions.

The local schema verification passed the required minimums for the ten application tables. The SQL audit extension was also present with related history records. The project demonstrates primary and foreign keys, uniqueness constraints, normalized repeating values, CRUD, joins, filtering, grouping, a summary view and a status change trigger. The website and SQL demonstration query the same MySQL database.

The project develops practical understanding of ER modeling, mapping cardinalities to foreign keys and bridge tables, removing multivalued attributes and connecting database transactions to user workflows. It also illustrates why ownership checks and constraints must support interface validation. Explainable matching shows how a computed result can be connected to stored evidence.

Future work can introduce schema migration tooling, automated regression tests, concurrency handling for duplicate submissions and a more complete event history. Resume analysis can add OCR for scanned documents, a controlled skill vocabulary and explicit evaluation against labeled examples. Matching can account for verified proficiency and job priorities, while keeping recruiter review available.

Further extensions include interview scheduling, notifications, analytics and accessibility improvements. Public deployment would benefit from file scanning, backups, monitoring and resume retention controls. MySQL remains the application database.

REFERENCES

The following technical references describe the database, framework and optional integrations used in this project. The project's own source and documentation identify its specific behavior and schema.

[1] Oracle, MySQL 8.4 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. Accessed: 3 October 2026.

[2] Pallets, Flask Documentation. [Online]. Available: https://flask.palletsprojects.com/en/stable/. Accessed: 3 October 2026.

[3] SQLAlchemy authors, SQLAlchemy 2.0 ORM Documentation. [Online]. Available: https://docs.sqlalchemy.org/en/20/orm/. Accessed: 3 October 2026.

[4] Artifex, PyMuPDF Documentation. [Online]. Available: https://pymupdf.readthedocs.io/en/latest/. Accessed: 3 October 2026.

[5] Google, Gemini API Documentation. [Online]. Available: https://ai.google.dev/gemini-api/docs. Accessed: 3 October 2026.

[6] Supabase, Storage Documentation. [Online]. Available: https://supabase.com/docs/guides/storage. Accessed: 3 October 2026.